RELUX WORKS · SWARMA

MCP из скилла: Curator настраивает сам

Дизайн, в котором скилл объявляет сервер и вход, а Curator готовит запуск

14 октября 2026 · черновик v0.1

Отдельная глава в формате книги Swarma. Дизайн по требованию владельца от 14 октября 2026 года; опирается на находки коллектива по MCP и на контракт csk для записи в нативные конфигурации. Это предложение архитектуры: ничего не собрано и не проверено на клиентах.

ПОМЕТКИ СТАТУСА

работает сегодня есть в коде и используется

решено владелец решил; кода может ещё не быть

спецификация описано в черновике спецификации модуля; кода нет

предложено предложение архитектуры, ждёт ревью или решения

открыто вопрос не решён

Содержание

MCP из скилла: Curator настраивает сам 3
- Что пишет автор скилла 3
- Как оператор подменяет сервер 4
- Что делает Curator 4
- Что видит оператор 6
- Пример целиком 6
- Что сохранено из выводов коллектива 7
- Что ещё открыто 7
- Источники 8

MCP из скилла: Curator настраивает сам

> Редакция: v0.1 (2026-10-14). Дизайн по требованию владельца. Он отвечает на четыре открытых вопроса из главы «MCP-серверы как зависимости скиллов» иначе, чем выигравший там дизайн N: закреплённая идентичность сервера нужна, вход объявляется в скилле, ключи в заголовках поддерживаются, Curator ставит сам. предложено

Требование владельца: скиллфайл сообщает, какой нужен MCP-сервер и какая у него аутентификация; Curator по максимуму настраивает всё сам, чтобы и запуск в профиле, и запуск из чекаута (как сейчас в csk) получали уже настроенный сервер; оператор может у себя подменить сервер, прошитый в скиллфайле, например через алиас. решено

Устройство в одну фразу: **скилл объявляет, оператор при желании подменяет, Curator составляет план, один раз спрашивает согласие и пишет конфигурацию в каждую цель запуска; входит в сервис по-прежнему клиент, а секретов Curator не видит.** предложено

Что пишет автор скилла

Требование остаётся там же, где сегодня: `dependencies.mcp_servers.<имя>`. К `hint`, `transport` и `required_in` добавляются два поля: **какой сервер** и **как в него входить**. предложено

```
dependencies:
  mcp_servers:
    figma:
      hint: "Figma Dev Mode MCP"
      server:
        endpoint: "https://mcp.figma.com/mcp"   # хостинг
      auth:
        kind: oauth                             # вход делает клиент
    linear:
      server:
        endpoint: "https://mcp.linear.app/mcp"
      auth:
        kind: bearer_env
        env: LINEAR_API_KEY                     # только имя переменной
    sqlite:
      server:
        package: { git: "https://github.com/acme/mcp-sqlite", range: "^1", name: sqlite }
      auth:
        kind: none
```

`server` — ровно одно из двух. предложено

- **`endpoint`** — сервер на хостинге: точный адрес HTTPS, схема, хост и путь. Обычный `http` разрешён только для `localhost`. Параметры запроса в адресе запрещены: через них слишком легко пронести токен.
- **`package`** — пакет объявления `agent-mcp.json`, тот же формат, что Curator уже использует в профилях. Из него берутся команда и аргументы для локального (stdio) сервера или адрес для удалённого.

`auth` — закрытый список видов, только с **именами** переменных и заголовков. Значений в нём не бывает. предложено

Вид	Что значит	Что пишет Curator в клиент
`none`	Входа нет	Только сервер

Вид	Что значит	Что пишет Curator в клиент
`oauth`	Вход через браузер, токен хранит клиент	Сервер; вход потом делает клиент
`bearer_env`	`Authorization: Bearer <значение переменной>`	Ссылку на переменную в синтаксисе клиента
`header_env`	Произвольный заголовок, например `X-Api-Key`, из переменной	То же для заголовка
`env`	Локальному серверу нужны переменные окружения	Список имён, передаваемых процессу

Схема закрыта: литеральные заголовки, значения, токены и пароли отклоняются при разборе. Поле живёт в новой версии манифеста (черновик v9 или следующая), потому что `mcpRequirement` в v5–8 закрыт и общий. Старые скиллы без `server` продолжают работать как сегодня: только проверка по имени. предложено

Имя MCP-сервера не должно совпадать с именем скилла: текущий резолвер не держит одноимённые скилл и MCP в одном замыкании (находка коллектива `f-cross-kind-name-collision`). предложено

Как оператор подменяет сервер

Подмена живёт **только у оператора**: в конфигурации машины Curator или в профиле, которым владеет оператор. Репозиторий может предложить подмену, но не применить её. Иначе чужой чекаут мог бы направить `LINEAR_API_KEY` на свой адрес. предложено

```
mcp_overrides:
  figma:
    use: figma-dev                  # алиас: взять уже настроенный у меня сервер с этим именем
  linear:
    server: { endpoint: "https://linear.internal.example/mcp" }
    auth:   { kind: bearer_env, env: LINEAR_INTERNAL_TOKEN }
  sqlite:
    skill: "acme/db-review"         # необязательно: подмена только для этого скилла
    server: { package: { git: "https://github.com/me/mcp-sqlite-fork", rev: "3f2c1a9", name:
sqlite } }
```

Возможны три вида подмены. предложено

- **Алиас** (`use`): требование закрывает сервер, который оператор уже настроил сам, руками, плагином или облачным коннектором. Curator в этом случае ничего не пишет, только запоминает привязку.
- **Другой сервер**: свой адрес или пакет и, при необходимости, свой способ входа. Дальше Curator ставит его так же, как поставил бы прошитый.
- **Отключение**: `off: true` — требование сознательно не удовлетворяется, и статус говорит об этом прямо.

Подмена по умолчанию действует для всех скиллов, которым нужно это имя; поле `skill` сужает её до одного. Подмена записывается в lock и всегда видна в статусе как «подменено оператором». предложено

Что делает Curator

1. Собирает требования

Curator обходит все скиллы профиля или проекта и собирает их MCP-требования. Затем применяет подмены и сводит одинаковые требования вместе: два скилла с одинаковым именем и одинаковой

идентичностью дают один сервер. Если два скилла хотят **разные** серверы под одним именем, Curator отказывает и предлагает подмену; молча он не выбирает. предложено

Пакет из диапазона версий разрешается до точного коммита и дайджеста и записывается в lock. Обновление пакета или смена адреса — это новый план. предложено

2. Составляет план и спрашивает один раз

План показывает каждую цель отдельно: какой файл, какой сервер, какой адрес или команда, какие **имена** переменных уйдут и на какой хост. Согласие нужно, когда сервер исполняет код (stdio) или когда переменная окружения уходит на адрес. Это правило D1 из csk, и оно сохраняется: токен из `GITHUB_TOKEN`, отправленный на адрес, выбранный чужим скиллом, — это маршрутизация учётных данных, её подтверждает человек. предложено

Согласие запоминается на пару «идентичность сервера + имена переменных + цель». Повторная установка того же плана проходит без вопросов. Новый адрес, новая переменная или новый получатель требуют согласия снова. предложено

Чтобы общий случай вообще проходил без вопросов, политика машины может заранее разрешить список серверов, например «эти пять адресов и эти пакеты с такими-то переменными». Для запуска агентом без человека годится только заранее одобренный план; флаг в скиллфайле или файл одобрения в репозитории сам себя не одобряет. предложено

3. Пишет конфигурацию в каждую цель

Запуск в профиле. Управляемый дом Curator. Здесь ничего нового: требования скиллов попадают в тот же набор, что и `requires.mcp` профиля, и материализуются по существующему пути §5.8 спецификации окружений. Это `--mcp-config` для Claude, `-p curator-mcp` для Codex и `OPENCODE_CONFIG` для OpenCode. Дом принадлежит Curator целиком, поэтому чужих записей в нём нет. спецификация

Запуск из чекаута. Нативные файлы проекта. Здесь Curator делает то же, что csk: пишет в `.mcp.json`, `.codex/config.toml`, `.cursor/mcp.json` и дальше по списку, с **владением по записи**. предложено

- Curator трогает только записи, которые сам создал и записал в свой закрытый реестр владения вне репозитория.
- Чужая запись с тем же именем — конфликт, даже если байты совпадают. Оператор решает его алиасом или явным принятием.
- Байты вне правки остаются точно такими же; комментарии не удаляются. Файл с синтаксисом, который нельзя сохранить, отклоняется до записи.
- Новый нативный файл по умолчанию добавляется в `.gitignore`; отслеживаемый файл правится на месте, и изменение видно в git.
- Пользовательские нативные файлы (`~/.claude.json` и подобные) — только по явной команде для глобальной установки.
- Нативный писатель никогда не пишет в управляемый дом, а управляемый путь — в нативные файлы.

Контракт csk для этой части уже продуман (решения D1–D8 и поправки A1–A7 первого среза). Предлагается не изобретать второй писатель, а сделать один общий контракт для Curator и csk. предложено

4. Пишет вход в синтаксисе каждого клиента

Для каждого вида входа у каждого клиента есть адаптер. Значение переменной Curator не читает: он пишет ссылку, а подставляет её клиент при запуске. предложено

Клиент	`bearer_env` / `header_env`	`oauth`
Claude Code	`headers` со ссылкой `${VAR}` в `.mcp.json` или в канале `--mcp-config`	`/mcp` в клиенте
Codex	`bearer_token_env_var` / `env_http_headers` в `config.toml`	`codex mcp login <имя>`
Cursor	`headers` со ссылкой `${env:VAR}`	вход в клиенте
OpenCode	`headers` со ссылкой `{env:VAR}`	вход в клиенте
Gemini CLI	`headers` со ссылкой на переменную в `settings.json`	вход в клиенте

Синтаксис ссылок нужно подтвердить на точных версиях клиентов до заявления о поддержке; таблица — направление, а не квалификация. Если клиент не умеет нужный вид входа, статус говорит «не поддерживается для этого клиента» и ничего не изображает. открыто

В профиле добавляется одно: имя переменной из `auth` автоматически попадает в план как разрешённое к передаче (`passable_env_names`), с тем же согласием. Сегодня управляемый HTTP-путь заголовки из переменных не передаёт; этот дизайн добавляет такую возможность в виде «только имена». предложено

5. Помогает войти, но токена не держит

Для `oauth` Curator после установки говорит, что сделать, и по просьбе может запустить **команду входа самого клиента** в правильном доме. Например, `curator mcp login figma --agent codex` запускает `codex mcp login figma` с окружением профиля, и токен ложится туда, откуда его потом возьмёт запуск профиля. Браузер, токен и его хранение остаются у клиента. предложено

Для `bearer_env` значение задаёт оператор: в окружении, а позже через брокер учётных данных платформы. Curator проверяет только, что переменная с таким **именем** задана, но не её значение. предложено

Проверок здоровья нет: тестовый запрос к серверу сам может отправить токен. Если оператор хочет знать, подключилось ли, Curator может по явной просьбе показать вывод собственной проверки клиента, например `claude mcp list`. предложено

Что видит оператор

Статус показывается по каждому требованию и каждой цели запуска, в двух измерениях: **настройка** и **вход**. предложено

```
figma   профиль dev / claude   настроен Curator (https://mcp.figma.com/mcp)   вход: oauth, не проверен
figma   чекаут / codex         подменено оператором → figma-dev                вход: у клиента
linear  профиль dev / codex    настроен Curator                                вход: LINEAR_API_KEY задана, значение не проверено
sqlite  чекаут / cursor        конфликт: чужая запись sqlite                   нужно решение оператора
```

Слова «работает» и «подключён» Curator не пишет никогда: он этого не знает. предложено

Пример целиком

1. Скилл `design-review` объявляет `figma` на хостинге с входом `oauth`.

2. `curator install` в проекте показывает план: Claude и Codex в чекауте, сервер `https://mcp.figma.com/mcp`, переменных нет. Удалённый сервер без переменных по правилу D1 ставится без вопроса, но в плане он виден.
3. Curator пишет запись `figma` в `.mcp.json` и `.codex/config.toml`, отмечает их своими и добавляет новые файлы в `.gitignore`.
4. Curator подсказывает: «войдите: `/mcp` в Claude, `codex mcp login figma`». Оператор входит один раз в каждом клиенте.
5. Тот же скилл в профиле `dev`: запись попадает в канал MCP профиля, вход делается один раз командой входа в доме профиля.
6. В компании свой прокси для Figma. Оператор пишет подмену `figma → https://figma-proxy.corp/mcp` с `bearer_env: FIGMA_PROXY_TOKEN`. Curator показывает новый план, где токен уходит на новый хост, спрашивает согласие и переписывает только свои записи.

Что сохранено из выводов коллектива

Владелец выбрал закреплённую в скилле идентичность и установку силами Curator. Коллектив оценивал похожую исправленную базу в 10–12 очков из 16 против 13 у N, в основном за новые поля и за то, что менеджер берёт на себя больше. Цена известна, но всё, что коллектив нашёл как опасное, здесь закрыто явно. предложено

- **Маршрутизация учётных данных** (`f-consent-binding`): переменная на адрес — только с согласием; подмена — только у оператора; репозиторий ничего не одобряет.
- **Граница секретов** (`f-secret-boundary`): ни значений, ни проверок здоровья, ни вывода сырых argv, URL и заголовков; чужие байты нативных файлов разбираются закрыто, наружу выходят только метаданные; отказ от проверки секретов не делает материализацию с секретом допустимой.
- **Пакет — не сервис** (`f-package-is-not-service`): идентичность — точный адрес или пакет, закреплённый в lock до коммита и дайджеста; «тот же сервер» означает совпадение с закреплением, а не с именем.
- **Вселенная запуска** (`f-launch-universe`): статус считается по каждой цели; строгий канал Claude, отключения OpenCode и записи только в управляемом доме учитываются.
- **Непрозрачный провайдер** (`f-opaque-provider`): плагин или облачный коннектор, которого Curator не видит, закрывается алиасом с пометкой «заявлено оператором».
- **Коллизия имён** (`f-cross-kind-name-collision`): разные имена у скилла и его MCP-сервера.

Что ещё открыто

- **Где живёт спецификация.** Поля манифеста — в curator-spec (CIP, запись решения, нормативный текст, векторы); общий писатель нативных файлов — один контракт для Curator и csk. Нужно решить, кто им владеет. открыто
- **Синтаксис ссылок на переменные** в каждом клиенте подтверждается на точных версиях; до этого клиент без подтверждения получает статус «не поддерживается». открыто
- **Общий домен доверия.** Все MCP-серверы одного запуска видят одно окружение клиента; изоляции между серверами этот дизайн не обещает. открыто
- **Вход в управляемом доме**: поведение хранения токенов OAuth у каждого клиента при подменённом доме нужно проверить. открыто
- **Windows**: локальные серверы через `.cmd`/`.bat` (`npx` и подобные) отклоняются, как в csk, до отдельной квалификации. открыто

Источники

- Глава «MCP-серверы как зависимости скиллов» в этой же папке и синтез коллектива `mcp-deps/SYNTHESIS.md` @ `9e534c0`; находки `mcp-deps/findings/`. **предложено**
- Контракт csk для записи в нативные файлы: `mcp-deps/inputs/csk-mcp-design-proposal-v1.md` и решения первого среза `mcp-deps/inputs/csk-slice1-decisions.md` (D1–D8, A1–A7). **решено**
- curator-spec: `protocol/core.md` §4 (`dependencies.mcp_servers`), `protocol/environments.md` §§2.2, 2.3, 5.8, 7.8, 10.3 (пакеты объявлений, материализация, `passable_env_names`), `profiles/manager.md` §6. **спецификация**